

Hafta 4 — Derinlemesine

⚠ BU DOSYA SINAV KAPSAMI DIŞINDADIR

Buradaki hiçbir konudan sınavda soru sorulmaz. Bu dosya, "peki bu nasıl çalışıyor?" diye soran öğrenci içindir. Sınav kapsamı `H04-ders-notu.md` dosyasıdır.

Dönem projesinde işinize yarayacak birkaç şey de buradadır (bölüm 3, bölüm 6).

1. Öncelik Sırası Nereden Geliyor?

Ders notu bölüm 2.2'de kaynak sırasını **Form** → **Rota** → **Sorgu** olarak verdik. Bu sıra sihirli değil;

`MvcOptions.ValueProviderFactories` listesinin **sırasıdır**.

Çatı varsayılan olarak şu fabrikaları bu sırayla kaydeder:

#	Fabrika	Okuduğu yer
1	<code>FormValueProviderFactory</code>	<code>Request.Form</code>
2	<code>RouteValueProviderFactory</code>	<code>RouteData.Values</code>
3	<code>QueryStringValueProviderFactory</code>	<code>Request.Query</code>
4	<code>JQueryFormValueProviderFactory</code>	jQuery'nin ürettiği <code>a[0].b</code> biçimi

Model binding, parametre için değer ararken listeyi **baştan sona** gezer ve **ilk bulduğunu** alır. Sıra buradan çıkıyor.

İsterseniz değiştirebilirsiniz — ama çok iyi bir sebebiniz yoksa yapmayın:

```
builder.Services.Configure<MvcOptions>(o =>
{
    // sorgu dizesini en öne almak: artık ?deger= form gövdesini EZER
    var sorgu = o.ValueProviderFactories
        .First(f => f is QueryStringValueProviderFactory);
    o.ValueProviderFactories.Remove(sorgu);
    o.ValueProviderFactories.Insert(0, sorgu);
});
```

⚠ Bunu yaparsanız projedeki **her** action'ın davranışı değişir ve hiçbir yerde yazmaz. Tek bir parametre için kaynağı zorlamak gerekiyorsa `[FromQuery]` kullanın.

`[FromQuery]` gibi öznitelikler bu listeyi **atlar**: parametreye tek bir `IValueProvider` bağlar.

2. StringValues ve ICollection

`ICollection["Ad"]` bir `string` değil, `StringValues` döndürür.

Sebebi: HTTP aynı adı **birden çok kez** taşıyabilir.

```
Etiket=roman&Etiket=klasik&Etiket=rusca
```

```
StringValues v = form["Etiket"];
v.Count;           // 3
v[0];              // "roman"
v.ToString();      // "roman,klasik,rusca" ← virgülle birleştirir
(string?)v;        // aynı: birleştirilmiş metin
```

 **Sessiz tuzak:** `form["Etiket"]` sonucunu doğrudan bir `string` e atarsanız üç değer **virgülle birleşmiş tek metne** dönüşür ve bunu fark etmeniz zor olur. Çoklu değer bekliyorsanız `StringValues` olarak tutun ya da model binding kullanıp `List<string>` 'e bağlayın.

`StringValues` ayrıca **boş** olabilir: alan gönderilmemişse `v.Count == 0` ve `v.ToString()` boş metindir. "Gönderilmedi" ile "boş gönderildi" ayrımını `form.ContainsKey("Ad")` ile yaparsınız.

3. Seçici Bağlama: [Bind], [BindNever], TryUpdateModelAsync

Ders notu bölüm 3.4 ViewModel'i önerdi. Alternatifler de var; hepsinin bir bedeli var.

`[BindNever]` — özellik düzeyinde kapatmak

```
public class Kitap
{
    public string Baslik { get; set; } = "";

    [BindNever] // model binding bu alana ASLA dokunmaz
    public bool Onayli { get; set; }
}
```

Avantajı: kural **entity'nin yanında** durur; yeni bir action yazan kişi kuralı taşımayı unutamaz. **Dezavantajı:** aynı entity bir yönetim ekranında *gerçekten* bağlanmak istiyorsa orada da çalışmaz; ayrı bir yol gerekir.

`[Bind]` — izin listesi

```
[HttpPost]
public IActionResult Ekle([Bind("Baslik,Yazar,Fiyat")] Kitap kitap) { ... }
```

Dezavantajı — ders notunda değinildi: entity'ye **Ro1** alanı eklediğinizde bu listeyi güncellemeniz gerekmez (yeni alan bağlanmaz, iyi) **ama** **Yazar** alanının adını değiştirdiğinizde liste sessizce eskir ve o alan artık bağlanmaz. Kırılğanlık burada: **yazım hatası derleme zamanında yakalanmaz.**

TryUpdateModelAsync — çalışma zamanında seçmek

```
var kitap = _depo.Bul(id!);
await TryUpdateModelAsync(kitap, "", k => k.Baslik, k => k.Yazar, k => k.Fiyat);
```

İzin verilen alanlar **lambda ile**, yani **derleyicinin denetlediği** biçimde yazılır. Alan adı değişirse kod **derlenmez**. Güncelleme (edit) senaryolarında en sağlam yöntem budur.

Özet tercih sırası: yeni kayıt için **ViewModel**; var olan kaydı güncellemek için **TryUpdateModelAsync** ; **[Bind("...")]** metin listesinden kaçının.

4. Özel Model Bağlayıcı (custom model binder)

Bazen gelen metnin nesneye çevrilmesi standart kurallara sığmaz. Örnek: tek bir alanda **"978-0-14-044913-6"** gibi bir ISBN gelsin ve bunu doğrulanmış bir **Isbn** değer nesnesine çevirmek isteyelim.

```
public class IsbnBinder : IModelBinder
{
    public Task BindModelAsync(ModelBindingContext ctx)
    {
        var ham = ctx.ValueProvider.GetValue(ctx.ModelName).FirstValue;

        if (string.IsNullOrEmpty(ham))
            return Task.CompletedTask; // boş: dokunma, null kalsın

        if (!Isbn.TryParse(ham, out var isbn))
        {
            ctx.ModelState.TryAddModelError(ctx.ModelName, "Geçersiz ISBN.");
            return Task.CompletedTask; // ⚠ istisna FİRLATMA
        }

        ctx.Result = ModelBindingResult.Success(isbn);
        return Task.CompletedTask;
    }
}
```

Bağlama:

```
[ModelBinder(BinderType = typeof(IsbnBinder))]
public Isbn? Isbn { get; set; }
```

Değişmeyen kural: bağlayıcı **istisna fırlatmaz**. Geçersiz girdi bir uygulama durumudur; hatayı `ModelState` 'e yazar ve kararı action'a bırakır. Ders notu bölüm 2.7'deki davranış tam olarak budur.

5. Kültüre Bağlı Çeviri: Ondalık Ayracı ve Tarih

Bu, Türkçe kullanan ekiplerin en sık karşılaştığı duvardır.

`decimal Fiyat` alanına kullanıcı `12,50` yazdı. Bağlanır mı?

Cevap: sunucunun kültürüne bağlı. Model binding, form ve sorgu değerlerini

`CultureInfo.CurrentCulture` ile çevirir:

Sunucu kültürü	12,50	12.50
tr-TR	✓ 12,50	✗ bağlanmaz (ya da 1250)
en-US	✗ bağlanmaz	✓ 12.50
InvariantCulture	✗	✓

Üstelik aynı uygulama **geliştirme makinenizde** çalışıp **sunucuda** çalışmayabilir.

İki yaklaşım:

1 — Kültürü açıkça sabitlemek (istenen davranış neyse onu seçin):

```
var kultur = new[] { new CultureInfo("tr-TR") };
app.UseRequestLocalization(new RequestLocalizationOptions
{
    DefaultRequestCulture = new RequestCulture("tr-TR"),
    SupportedCultures = kultur,
    SupportedUICultures = kultur
});
```

2 — Sayıyı HTML tarafında standartlaştırmak: `<input type="number">` kullanın. Tarayıcı değeri **noktalı** (invariant) biçimde gönderir; kullanıcı ekranda virgül görsen bile.



Rota ve sorgu dizesi için ayrı bir not: ASP.NET Core rota değerlerini **invariant** kültürle çevirir, form değerlerini **CurrentCulture** ile. Yani aynı sayı `/Urun/Fiyat/12.50` yolunda çalışıp form gövdesinde çalışmayabilir. Tarih alanlarında bu fark baş ağrıtır; tarihleri `<input type="date">` ile alıp `yyyy-MM-dd` biçiminde göndermek en sağlamdır.

6. PRG Sonrası Mesaj Taşımak: TempData

PRG'nin bir yan etkisi vardır: yönlendirme **yeni bir istektir**, dolayısıyla `ViewData` / `ViewBag` ile taşıdığınız her şey kaybolur.

```
[HttpPost]
public IActionResult Ekle(KitapEkleVM girdi)
{
    ...
    ViewData["Mesaj"] = "Eklendi";           // ✗ yönlendirmeden sonra YOK
    TempData["Mesaj"] = "Eklendi";          // ✔ bir sonraki isteğe kadar yaşar
    return RedirectToAction("Index");
}
```

`TempData` değeri **bir kez okunana kadar** saklar, sonra siler. Varsayılan sağlayıcı bir **çerezdir** (`CookieTempDataProvider`), yani oturum gerekmez.

Taşıyıcı	Ömrü	Ne zaman
<code>ViewData</code> / <code>ViewBag</code>	Tek istek	Controller → görünüm
<code>TempData</code>	Bir sonraki isteğe kadar	PRG sonrası mesaj
<code>Session</code>	Oturum boyunca	Sepet, çok adımlı form

➡ SOON Üçünün tamamı ve çerez/oturum ayrımı **Hafta 7**'nin konusudur. Burada PRG'nin pratik bir eksiğini kapattığı için var.

⚠ `TempData` 'ya **büyük nesne koymayın**: çerez sağlayıcıda değer serileştirilip **her istekte** tarayıcıya gidip gelir. Kısa bir mesaj için uygundur, liste için değil.

7. CSRF — Neden Bu Hafta Yok?

Formunuzu başka bir sitedeki görünmez bir form **sizin adınıza** gönderebilir: kullanıcı sitenizde oturum açmışsa tarayıcı çerezi otomatik ekler ve sunucu isteği meşru sanır. Bu saldırının adı **CSRF (Cross-Site Request Forgery)**.

Savunma, forma sunucunun ürettiği gizli bir **anti-forgery token** koymak ve POST'ta doğrulamaktır:

```
<form method="post">
    @Html.AntiForgeryToken()
    ...
</form>
```

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Ekle(KitapEkleVM girdi) { ... }
```

Bu hafta konu edilmedi, çünkü **Tag Helper ile yazılan form bu token'ı kendiliğinden ekler** — yani asıl anlatım yeri Hafta 5'tir. Bu haftanın formlarını **elle** yazdığımız için token yok; ev görevinizde de beklenmiyor.

✖ Yine de bu haftanın ilkesiyle aynı aileden: *istemciden gelen hiçbir veri güvenilir değildir* — isteğin **kaynağı** da dahil.

Kaynak: [Prevent XSRF/CSRF attacks in ASP.NET Core](#)

8. [FromBody] ve JSON — Hafta 12'nin Ön Tadı

Form gövdesi `ad=değer&...` biçimindeydi. Web API'lerde gövde genellikle **JSON**'dur ve onu okuyan öznitelik `[FromBody]` 'dir.

```
[HttpPost]
public IActionResult Ekle([FromBody] KitapEkleVM girdi) { ... }
```

```
POST /api/kitap
Content-Type: application/json

{ "baslik": "Suc ve Ceza", "fiyat": 120 }
```

Üç fark:

	Form bağlama	[FromBody]
Gövde biçimi	<code>x-www-form-urlencoded</code>	<code>application/json</code>
Okuyan	Value provider'lar	Input formatter (JSON çözücü)
Kaç kez	Kaynak sırası boyunca	Action'da yalnızca bir parametre
Ad eşleşmesi	Tam ad	Varsayılan olarak büyük/küçük harf duyarsız

`[FromBody]` 'nin tek kez kullanılabilmesinin sebebi teknik: gövde **ileriye doğru okunan bir akıştır**; ikinci parametre için baştan okunamaz. (Bu hafta `/Form` sayfasında gövdeyi iki kez okuyabildik çünkü `EnableBuffering()` çağırdık — yalnızca öğretim amaçlı bir istisna.)

9. Daha İleri Gitmek İsteyenlere

Konu	Nerede
Model binding — tam referans	https://learn.microsoft.com/aspnet/core/mvc/models/model-binding?view=aspnetcore-10.0

Özel model bağlayıcı	https://learn.microsoft.com/aspnet/core/mvc/advanced/custom-model-binding?view=aspnetcore-10.0
CSRF koruması	https://learn.microsoft.com/aspnet/core/security/anti-request-forgery?view=aspnetcore-10.0
İstek yerelleştirme (kültür)	https://learn.microsoft.com/aspnet/core/fundamentals/localization?view=aspnetcore-10.0
Dosya yükleme	https://learn.microsoft.com/aspnet/core/mvc/models/file-uploads?view=aspnetcore-10.0
Form verisi gönderme (MDN)	https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Sending_and_retrieving_form_data

⚠️ Bağlantıların sonundaki `?view=aspnetcore-10.0` **kasıtlıdır**. Sürüm sabitlenmezse arama motoru sizi eski sürüme düşürür ve `Startup.cs` 'li örneklerle karşılaşabilirsiniz.